MODEL REPORT

SplitSnap

Fine-tuned receipt extraction + correction-aware bill splitting

Summary. SplitSnap turns a receipt photo into structured bill JSON using a self-fine-tuned Donut model (naver-clova-ix/donut-base). Fine-tuning uses SROIE first, then CORD for the final SplitSnap schema, with a second charge-focused CORD epoch. The app loads the trained checkpoint from Hugging Face (prakursh/splitsnap-donut) via hf_model_repo.txt, treats extraction as an editable draft, and uses deterministic splitting logic after user correction.

1. System overview

Receipt image	Fine-tuned Donut	Bill JSON	Review & correct	Assign items	Split + explain

How it works. The model generates JSON with items (name, quantity, unit price, total, optional discount), charges, subtotal and grand total. Inference uses greedy decoding with a 768-token limit. If output is invalid JSON, the app tries the outermost { ... } block; if that still fails, it opens an empty editable bill instead of blocking the workflow.

Working end-to-end extraction flow

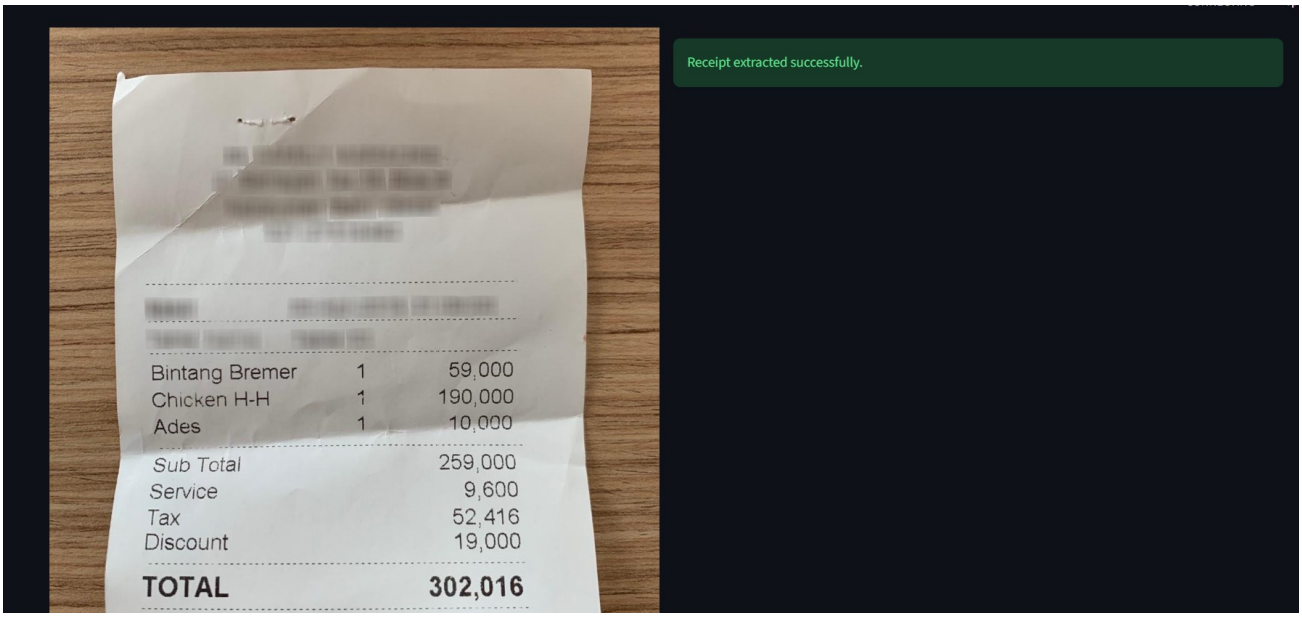


Figure 1. A CORD receipt is extracted successfully in the Streamlit app before correction and splitting.

Headline results on 94 held-out CORD test receipts

93.62%	91.49%	85.11%
Valid JSON	Exact grand total	Charge F1

TRAINING

2. Model training and preprocessing

Base model. naver-clova-ix/donut-base. The fine-tune is two-stage at the dataset level: SROIE for auxiliary receipt understanding, then CORD for the final SplitSnap schema. Within CORD, a second epoch oversamples charge-bearing receipts to correct low charge recall.

Training step	Dataset	Purpose	Key outcome
Stage 1	SROIE	Auxiliary receipt entity learning	Validation loss 5.343 -> 0.131
Stage 2 - CORD epoch 1	CORD	Learn items / charges / subtotal / total JSON	Val loss 0.055; valid JSON 89.58%
Stage 2 - CORD epoch 2	CORD, charge-focused	Oversample tax / service / discount receipts	Charge F1 32.65% -> 91.26%

Preprocessing choices

- CORD annotations were normalized into one schema: items, charges, subtotal and grand_total.
- Mixed formats such as 75,000; 25.000; 8.500,00 and 1,234.56 were normalized before training.
- Discounts are stored as negative amounts; ambiguous or mathematically inconsistent receipts were filtered.
- Targets longer than 768 tokens were excluded rather than truncated (5 of 769 CORD training examples).
- Images were resized to 1280 x 960, matching the processor configuration used by the saved checkpoint.

Training configuration and dataset sizes

Setting	Value	Setting	Value
Image size	1280 x 960	CORD train / val / test	764 / 96 / 94
CORD max length	768 tokens	SROIE train / val	886 / 100
Batch	1 physical x 4 accumulation	Stage 1 LR	5e-5
Precision / optimizer	FP16 / AdamW	CORD epoch 1 / 2 LR	3e-5 / 1e-5
Peak GPU memory	5.41 GB (Tesla T4)	Inference	Greedy, max 768 tokens

Why the charge-focused epoch mattered

Validation metric	Epoch 1	Epoch 2	Change
Valid JSON	89.58%	98.96%	+9.38 pp
Exact grand total	84.38%	96.88%	+12.50 pp
Charge recall	20.00%	92.16%	+72.16 pp
Charge F1	32.65%	91.26%	+58.61 pp
Exact charge set	58.33%	92.71%	+34.38 pp
Exact item-name list	59.38%	75.00%	+15.62 pp

Epoch 1 had high charge precision (88.89%) but only 20% recall. Charge-focused oversampling made the model emit taxes, service charges and discounts reliably enough for downstream splitting.

EVALUATION

3. Final evaluation

Held-out test set. The selected checkpoint was evaluated once on 94 untouched CORD test receipts. Metrics are generation-based, not just teacher-forced loss. Validation results are shown beside the test set to expose the generalization gap.

Metric	Validation (96)	Test (94)	Gap
Valid JSON	98.96%	93.62%	-5.34 pp
Exact grand total	96.88%	91.49%	-5.39 pp
Exact subtotal	90.62%	85.11%	-5.51 pp
Exact charge set	92.71%	85.11%	-7.60 pp
Charge precision	90.38%	83.33%	-7.05 pp
Charge recall	92.16%	86.96%	-5.20 pp
Charge F1	91.26%	85.11%	-6.15 pp
Exact item-name list	75.00%	62.77%	-12.23 pp

Reading the table: *totals and charges transfer reasonably well, while exact item-name lists are the weakest metric. Because item names are free text, one wrong character fails the whole list; this is why correction is a core product step.*

Correction layer: model output is an editable draft

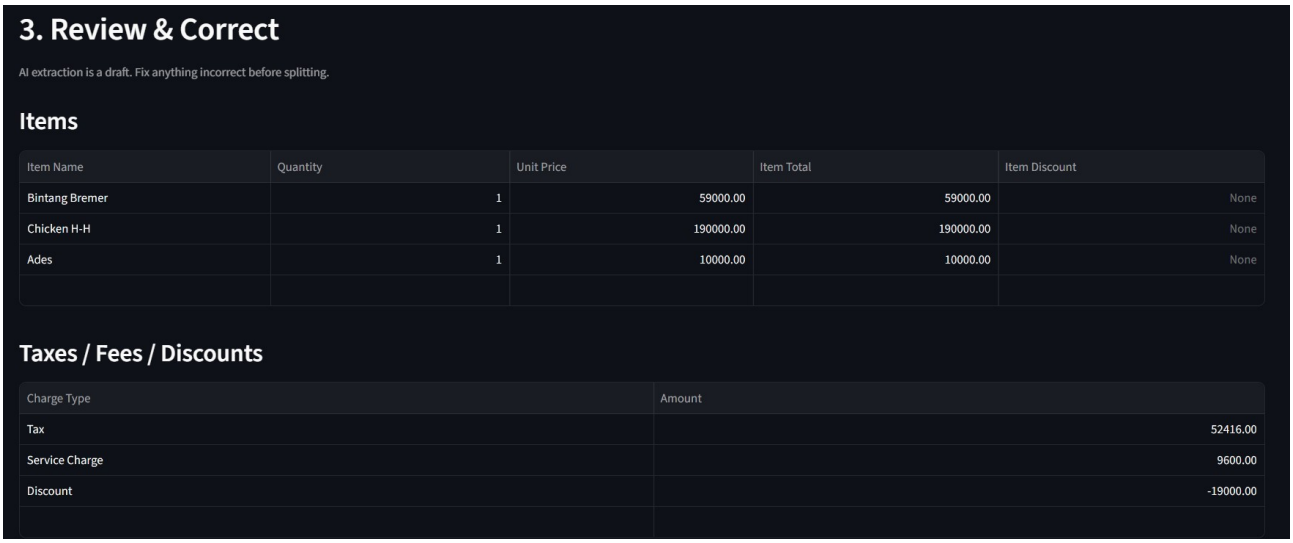


Figure 2. Users can edit items, charges, discounts, subtotal and grand total before splitting.

- Items editor: name, quantity, unit price, item total and item-level discount.
- Charges editor: Tax, Service Charge, Discount or Other Adjustment; discounts are represented as negative amounts.
- Subtotal and grand total are editable; corrections are committed with Save Corrections.
- After correction, the consistency checker compares reconstructed bill value with the printed grand total before splitting.

PRODUCT RELIABILITY

4. Safeguards and deterministic split logic

Consistency check

- Uses printed subtotal when available; otherwise rebuilds the base from item totals plus item-level discounts.
- Compares base + bill-level charges with grand total using a 0.02 tolerance.
- A negative gap is flagged as a likely missing discount; a positive gap as a likely missing tax, fee or other surcharge.
- Shows calculated total, receipt total and the unexplained difference.
- Invalid JSON never blocks the workflow; manual correction remains available.

Split engine

- Every item must be assigned; shared items are divided equally among selected people.
- Item-level discounts are applied before an item is split.
- Bill-level taxes, service charges and discounts are allocated by each person's share of assigned item value.
- Split calculations use Decimal with ROUND_HALF_UP; the final person receives the rounding remainder so allocated amounts stay exact.
- The final split is checked against grand total and explained per person; optional manual overrides are also rechecked.

Reproducibility matches the repository

Model source	hf_model_repo.txt -> prakursh/splitsnap-donut
Loading	app.py calls DonutProcessor.from_pretrained and VisionEncoderDecoderModel.from_pretrained
Key pinned versions	torch 2.14.1; transformers 5.16.1; streamlit 1.65.0; pandas 3.0.6; huggingface_hub 1.33.0
Run command	python -m streamlit run app.py
Code + metrics	github.com/Prakursh/splitSnap

Worked example

Alice has 700 of assigned items and Bob 300; with tax 100, Alice receives 70 of the tax and Bob 30, so the totals are 770 and 330. A bill-level discount of -50 is allocated the same way (-35 and -15).

Limitations and next steps

- Domain shift remains the main model limitation: unfamiliar layouts, currencies, fonts or poor image quality can produce wrong item names or malformed JSON.
- JSON recovery is intentionally minimal (outermost-brace extraction), not a full repair system.
- Item totals are not cross-checked against quantity x unit price; the implemented consistency check is bill-level.
- format_money currently renders the rupee symbol for all currencies, including non-INR CORD receipts.
- The consistency checker detects a mismatch but cannot identify which extracted field is wrong.
- Next steps: Indian-receipt fine-tuning, blur/perspective augmentation, field confidence, out-of-distribution detection, automatic JSON repair and a more mobile-friendly UI.

Conclusion: ML extracts the draft; deterministic software and user correction make the money trustworthy.